

Assignment 1

Assigned on Monday October 20. You should upload your program to uzak.etu.edu.tr by Sunday, November 9 (any time up to midnight). When uploading do not compress your source directory, upload each file in your source directory separately, and do NOT use packages. Remember that your code should be fully documented—read the syllabus. I also once again remind you to read the academic honesty policy stated in the syllabus.

1 Background and Purpose

In this assignment, you will implement a Public Health Alert Notification System. Global health organizations such as the World Health Organization (WHO), Centers for Disease Control and Prevention (CDC), and European Centre for Disease Prevention and Control (ECDC) rely on real-time digital platforms to monitor and track disease outbreaks. This project simulates a simplified version of such a system: it processes timestamped **health incidents** (disease events) and **user requests** (watcher requests) in a discrete-time event simulation.

The aim is to demonstrate how data structures, temporal event management, and spatial proximity calculations can be integrated to form an automated early-warning system for public health.

2 System Overview

You will implement a program called `HealthAlertNotification` in Java. The program reads and processes two static input files in chronological order: (In real life these would have been realtime data streams, but you will implement a discrete time simulation using static files.)

1. **health-file** – contains reports of health incidents (disease outbreaks).
2. **watcher-file** – contains user requests to register (add) or remove (delete) watchers, as well as queries regarding health incidents.

The program can be viewed as a discrete event simulation, where events could be either health incidents or user requests; events take place at discrete times, denoted by their timestamps; and are processed in the order that they occur. You may assume that each file contains the events in increasing time order (earliest event first). You may assume the timestamp given is the hours passed since the start of the time (time=0).

You can implement the discrete event simulation as follows: Assume that your simulation starts at time 0 (initialize time to 0). Your program should run in a loop, each iteration of which corresponds to a time slice of 1 hour. At each iteration, the simulation checks if there is an event at that time (note that events are read from two separate files), and if there is, it processes the event, otherwise it proceeds to the next iteration.

3 Input Files

3.1 Watcher File Format

Each line describes a watcher event. The line starts with the timestamp of the request event. It is followed by one of *add*, *delete*, *query-highest*, *query-disease*, *query-region* requests. The add request will be followed by two numbers denoting latitude and longitude of the user's location followed by the watcher's name; the delete request will only be followed by the watcher's name. Note that the name is a unique identifier for a watcher.

```
<time> add <latitude> <longitude> <name>
<time> delete <name>
```

The information and format for each query request will be explained later.

Example Watcher File:

```
0 add -122.4 37.8 user1_WHO
1 add -118.3 34.0 user2_CDC
3 query-highest
5 add -121.8 38.2 user3_ECDC
6 add -123.1 37.9 user4_WHO
8 delete Taylor
10 query-disease Influenza
13 query-region -122.0 38.0 100
```

3.2 Health File Format (One Line per Health Incident)

Each line of the health file corresponds to one health incident. Note that locations and reporting agency names use underscores for easy parsing. Attributes are separated by spaces.

```
<time> <disease> <latitude> <longitude> <location(with_underscores)>
<infection_rate> <population_affected> <severity> <reporting_agency(with_underscores)>
```

Example:

```
6 Influenza_A -122.4194 37.7749 Downtown_San_Francisco_CA 1.8 230 3.9
  SF_General_Hospital
15 Norovirus -118.8353 37.493 Mammoth_Lakes_CA 1.2 140 4.6 Mono_County_Clinic
22 COVID-19 -122.3321 47.6062 Seattle_WA 2.3 560 4.9 UW_Health
```

4 Data Structures

You must use the `LinkedPositionalList` and `LinkedQueue` classes as given under Piazza Resources. You may NOT use built-in Java collections.

- (1) **watcher-list:** The first is a `LinkedPositionalList` of watchers who have registered for health incident notifications. It maintains registered watchers in the order of registration. Users will be deleted if such query request exists.

- (2) **incident-queue:** The second data structure is a queue to store the list of recent health incidents, in order of arrival. It will only retain the health incident records from the most recent 6 hours time period. As simulation time moves on, you will delete records with timestamps older than 6 hours from the queue.
- (3) **severity-ordered-list:** The third data structure is another `LinkedPositionalList`, which stores these same health incident records (again, removing records that are more than 6 hours old), however it maintains incidents ordered by severity (descending order).

As health incident records arrive or expire, they are added to or removed from both the `incident-queue` and the `severity-ordered-list`. Since the queue is ordered by time, it is easy to find which records are old. But you will also need to have an efficient way to find those records in the `severity-ordered-list`. To do that make sure that a health incident record (entry) stores the `Position` which contains itself in the `severity-ordered-list`. This way as soon as you delete an entry from the queue, you can determine which `Position` will be removed from the `severity-ordered-list` as well.

5 Event Simulation and Distance Calculation

Your two input files each have timestamped records. As the time progresses in your discrete time simulation, you will process the various events. **Note that user events have higher precedence (processed first) in case two events happen at the same time.**

- **User Events:** When you process an add request, a new watcher with the given information is added to the watcher-list and you print a suitable message to standard output (See below for output format). When you process a new watcher delete request, the watcher is removed from the watcher-list and you print a suitable message to standard output.

In addition, the following query types must be supported:

- `query-highest`: Displays information about the most severe incident within the past six hours.
- `query-disease <disease>`: Lists all incidents of the given disease within the past six hours.
- `query-region <latitude> <longitude> <radius>`: Lists incidents within a specified `<radius>`(km) of the coordinate given as (`<latitude>`, `<longitude>`) within the past six hours.

- **Health Incident Events:** When a new incident occurs, insert it into the `severity-ordered-list` (ordered by severity) and to the `incident-queue` (ordered by time). Output a message to standard output as when a new incident happens (See below for output format).

Remember to check the timestamp for the oldest incident(s), and remove any that are more than six hours old. To do this, you will look at the front of the queue. (As also explained above, for any incidents that must be removed, you will need to find their `Position` in the `severity-ordered-list`. You are NOT allowed to do this using an exhaustive search of the

severity-ordered-list. Instead, you must store in the health incident record the Position of itself in the severity-ordered-list (severity-ordered-list'te bu health incident'in kendisini içeren Position).

In addition to updating the incident-queue and the severity-ordered-list, you will also check against the watcher-list when you see a new health incident. You will output notification messages for each watcher that is “close enough” to the health incident. Each watcher record contains an (latitude, longitude) position, as does the health incident record.

A watcher should receive a notification if:

$$\text{Distance} < 2 \times \text{Severity}$$

The distance between two coordinates (lat_1, lon_1) and (lat_2, lon_2) can be computed using the Euclidean approximation:

$$d = \sqrt{(lat_1 - lat_2)^2 + (lon_1 - lon_2)^2}$$

In case of a notification, a suitable message sent to standard output (See below for output format).

6 Output Format

You MUST follow the following output format.

- When a watcher name is added to the watcher-list, you should print out:

```
<name> is added to the watcher-list
```

- When you process a new watcher delete request, you should print out:

```
<name> is removed from the watcher-list
```

- When you process a query-highest query, you should print out:

```
Most severe health incident in past 6 hours:  
(Disease: <disease>) Severity: <severity> at <location>
```

For example:

```
Most severe health incident in past 6 hours:  
(Disease: Norovirus) Severity: 4.6 at Mammoth_Lakes_CA
```

Note: if the severity-ordered-list is empty, you should print out:

```
No records
```

- When you process a query-disease <disease> query, you should print out all incidents of the given disease within the past six hours, each incident on a separate line as:

```
(Disease: <disease>) Severity: <severity> at <location>
```

- When you process a query-region <latitude> <longitude> <radius>: query, you should print out all incidents within <radius> of (<latitude>, <longitude>) that happened within the past six hours. Each incident must be printed on a separate line as:

```
(Disease: <disease>) Severity: <severity> at <location>
```

- If the --all option has been given, then you should print the following message each time that a health incident is added to the incident-queue/severity-ordered-list:

```
(Disease: <disease>) at <location> is inserted into incident-queue
```

For example:

```
(Disease: Norovirus) at Mammoth_Lakes_CA is inserted into incident-queue
```

- When a new health incident is added to the incident-queue/severity-ordered-list, you should print a single line for each watcher that is within the notification distance. Each such line should appear as:

```
(Disease: <disease>) at <location> is close to <name>
```

For example:

```
(Disease: Norovirus) at Mammoth_Lakes_CA is close to user1_WHO
```

7 Program Invocation

The program will be invoked from the command line as:

```
java HealthAlertNotification [--all] <watcherFile> <healthFile>
```

If the optional `--all` flag is specified, the program must output messages for all health incident insertions. Otherwise, when the `--all` option is not given, you will only be outputting notification messages (messages about health incidents that are close to a watcher) and messages related to the user requests.

DO NOT hardcode filenames within your Java code.

8 Submissions

Upload to `uzak.etu.tr` the following files:

1. **README:** The plain text file `README` that contains the following information:

Your name.

How to compile your code.

How to run your program.

Known Bugs and Limitations: List any known bugs, deficiencies, or limitations with respect to the project specifications.

2. **Source files:** all `.java` files